App Development Project Report

MediCare: Hospital Management System

1. Student Details

Field	Details
Name	Shrishti Gupta
Roll Number	23f2004336
Email	23f2004336@ds.study.iitm.ac.in
Program	IIT Madras BS Data Science & Applications

About Me: I am an undergraduate student in the IIT Madras BS Data Science & Applications program. I enjoy building websites that solve real-world problems. Building this complete web application from scratch was a big challenge, but it helped me learn how to connect a frontend interface with a backend database securely.

2. Project Details

Project Title: MediCare: Hospital Management System

The Problem

Many hospitals still rely on paper records or disconnected software that is hard to use. This causes lost files, double-booked appointments, and doctors struggling to track patient schedules and histories.

My Solution

I built a completely digital, role-based web application that brings hospital administration, doctors, and patients together in one platform. Patients can book appointments from their phones, doctors can set their exact working hours and record treatments, and admins can manage the whole hospital without any paperwork.

3. AI / LLM Declaration

I used Google Gemini to assist me while building this project. I designed the database, planned the features, and wrote the main code myself. However, when my code crashed or I got stuck on a bug (especially connecting Vue.js with Flask), I used AI to help find the mistake and understand how to fix it.

Extent of AI/LLM usage: approximately 20% of the project, used strictly as a debugging and tutoring tool.

4. Technologies Used

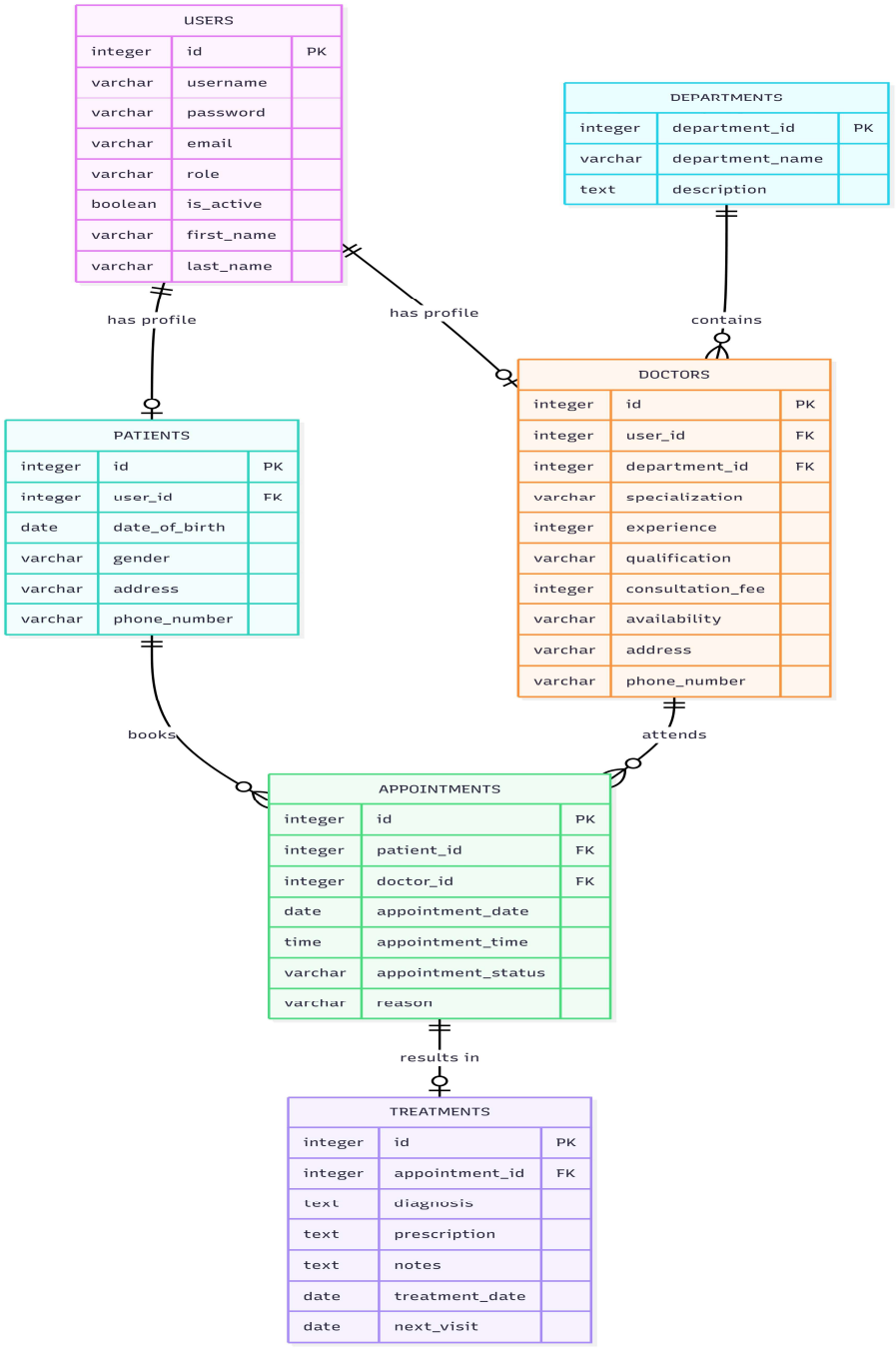
Technology	Purpose
Flask (Python)	Core backend framework: handles all API routes, authentication logic, and database operations.
Vue.js 3	Frontend SPA framework: makes the UI feel fast with reactive components and client-side routing.
Flask-SQLAlchemy	ORM to define and interact with the database using Python models instead of raw SQL.
SQLite	Lightweight local database for storing all users, appointments, treatments, and hospital data.
Flask-JWT-Extended	Token-based authentication: issues JWT tokens on login for secure role-based access control.
Bootstrap 5	Frontend styling framework: provides responsive layouts, modals, tables, and form components.
Redis	In-memory store used as Celery message broker and for Flask-Caching API response caching.
Celery	Distributed task queue for running background jobs: daily reminders, monthly reports, CSV exports.
Flask-Caching	Caches frequently accessed API responses (e.g. doctor list) with configurable expiry times.
Axios	HTTP client used in Vue.js to make API calls to the Flask backend.

5. Database Design (ER Diagram)

The database is organized into 6 tables designed to avoid data duplication and maintain clean relationships:

- Users: Master list of all users (admin, doctor, patient) with role, credentials, and profile fields.
- Patients: Extends Users with patient-specific details like date of birth, gender, address, and phone number.
- Doctors: Extends Users with specialization, qualification, experience, consultation fee, and 7-day availability schedule.
- Departments: Hospital departments that group doctors by medical specialization.
- Appointments: Connects a Patient and Doctor with a date, time, status (Booked/Confirmed/Completed/Cancelled), and reason.
- Treatments: Stores the doctor's diagnosis, prescription, notes, and next visit date after a completed appointment.

ER Diagram:



6. Main Features of the Application

The application is built as a two-part system: Vue.js handles the user interface and Flask handles secure data processing. Features are organized by role:

Admin Features

- Dashboard showing total doctors, patients, and appointments at a glance.
- Add, update, and remove doctor profiles including specialization, qualification, fee, and contact details.

- View and manage all appointments across the hospital.
- Search patients by name, email, or contact information.
- Remove or blacklist patients and doctors from the system.
- Manually trigger daily reminders and monthly report jobs from the dashboard.

Doctor Features

- Private dashboard showing upcoming (Booked/Confirmed) and past (Completed/Cancelled) appointments.
- Set a 7-day availability schedule with custom start time, end time, and off-duty days.
- Confirm or cancel patient appointments.
- Fill a treatment form with diagnosis, prescription, doctor notes, and next visit date to mark an appointment as completed.
- View and edit the full medical history of any assigned patient.

Patient Features

- Self-registration and login with secure password hashing.
- Search for doctors by name or specialization with a dropdown filter.
- View doctor availability for the next 7 days before booking.
- Book appointments with conflict prevention (no double-booking same doctor, date, and time).
- Reschedule or cancel existing appointments.
- View appointment history with status, diagnosis, prescription, doctor notes, and next visit date.
- Export full treatment history as a CSV file via an async background job (download link sent by email).
- Edit personal profile including name, gender, date of birth, address, and phone number.

Background Jobs (Celery + Redis)

- Daily Reminder (Scheduled): Runs every day at 8:00 AM IST. Sends an email reminder to every patient with an appointment booked for that day.
- Monthly Doctor Report (Scheduled): Runs on the 1st of every month. Generates an HTML email report for each doctor showing all completed appointments and treatments for that month.
- CSV Export (User-Triggered Async): Patient triggers export from their dashboard. A Celery task generates the CSV with full treatment history and emails a download link on completion.

Security

- Passwords are hashed using Werkzeug's generate_password_hash: never stored as plain text.
- JWT tokens are issued on login and verified on every protected API route.
- Role-based access control prevents patients from accessing admin or doctor routes.

- Doctors can only edit treatment records belonging to their own appointments.

7. API Resource Endpoints

Endpoint	Method	Description
/register	POST	Patient self-registration
/login	POST	Login for all roles, returns JWT token
/admin/stats	GET	Dashboard stats: total doctors, patients, appointments
/admin/doctors	GET/POST	List all doctors / Add a new doctor
/admin/update_doctor/<id>	PUT	Update an existing doctor's profile
/admin/patients	GET	List all patients (admin only)
/admin/patient/<id>	DELETE/PUT	Remove or edit a patient record
/admin/appointments	GET	View all hospital appointments
/admin/trigger/daily	POST	Manually trigger daily reminder job
/admin/trigger/monthly	POST	Manually trigger monthly report job
/doctor/schedule	GET/PUT	Get or update doctor 7-day availability
/doctor/appointments	GET	Get all appointments for logged-in doctor
/doctor/appointment/<id>	PUT	Update appointment status (Confirmed/Cancelled)
/doctor/treat_patient	POST	Save diagnosis, prescription, notes for an appointment
/doctor/patient_history/<id>	GET	View full treatment history of a patient
/doctor/update_treatment/<id>	PUT	Edit an existing treatment record
/patient/doctors	GET	List all doctors with availability info
/patient/my_appointments	GET	Get logged-in patient's appointments with treatment details
/patient/book	POST	Book a new appointment with conflict prevention
/patient/appointment/<id>/cancel	PUT	Cancel a booked appointment
/patient/appointment/<id>/reschedule	PUT	Reschedule an existing appointment

/patient/profile	GET/PUT	View or update patient profile
/patient/export_history	POST	Trigger async CSV export of treatment history
/doctors	GET	Public cached doctor list

8. Video Presentation

Drive Link: https://drive.google.com/file/d/1gj1xl3O2vo4OZG-bRU5oOdfH0OK_6oxd/view?usp=sharing